

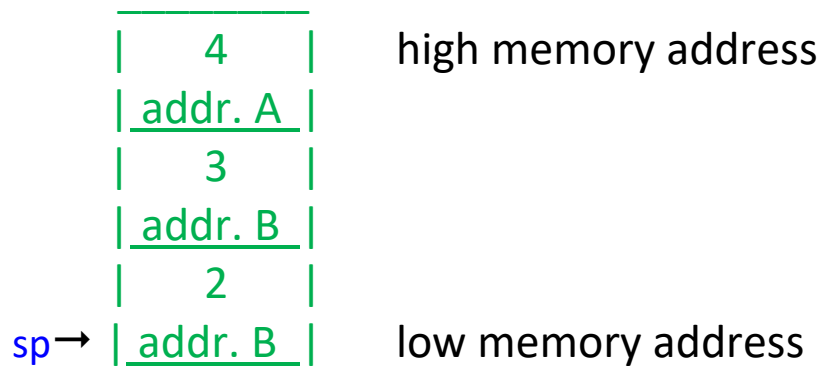
example of a recursive procedure: factorial, using formula $N! = N \times (N-1)!$ (but note, would never compute factorial recursively this way in practice!)

```
# portion of main program
...
mv a0, s0          # assume number we want factorial of
jal ra, fac        # is in s0
li a7, SYS_printInt
ecall              # print the result
...

# procedure code
fac: li t0, 1
    bgt a0, t0, notbase # base case is n = 0 or 1
    li a0, 1            # if it is the base case, return 1
    jalr zero, 0(ra)
notbase: addi sp, sp, -8 # allocate 2 words for frame
                                # (but in typical practice some space
                                # would be caller-allocated)
    sw a0, 4(sp)        # store our parameter ("n")
    sw ra, 0(sp)        # store our return address
    addi a0, a0, -1     # create parameter for recursive call
    jal ra, fac         # finds (n-1)!
    lw t0, 4(sp)        # retrieve n from procedure call frame
    mul a0, a0, t0      # n! = n x (n-1)!
    lw ra, 0(sp)        # restore our return address
    addi sp, sp, 8      # remove procedure call frame
    jalr zero, 0(ra)
```

(note: for "mul", need extension `m': "ASFLAGS=-march=rv32im")

suppose main program calls “fac” with parameter 4
stack contents during the last of the recursive calls
(to “fac” with parameter 1) would be as follows:



here the name A represents the memory address of the instruction following the “jal ra, fac” instruction in the main program, and the name B represents the memory address of the instruction following the “jal ra, fac” instruction inside the procedure

the horizontal lines delimit the portions of the stack used for each call frame

often a special purpose register, the **frame pointer (fp)**, is used to store the address of the first word of the current call frame, but we will not adopt this usage

How do other machine/assembly languages compare to RISC-V?

(reference: text page 125, [Section 2.24](#))

basic styles of machine language

“load-store” (or “register-register”) style

- this is what RISC-V uses
- arithmetic/logical instructions have three operands, all of which must be registers except possibly for the last which can be a constant

“accumulator” style

- only a single register (“accumulator”)
- accumulator is an implicit operand
- e.g. to implement “ $a = b + c$ ”:

load b

add c

store a

“register-memory” style

- at most one of the two given operands can be in memory; one operand (which must be a register) both a source and the destination
- e.g. to implement “ $a = b + c$ ”:

load R1, b

add R1, c

store R1, a

“memory-memory” style

- three operands, each of which can be in memory or be a register
- e.g. to implement “ $a = b + c$ ”: add a, b, c

“stack” style

- hardware-supported stack instead of registers
- e.g. to implement “ $a = b + c$ ”:

push b

push c

add

pop a

addressing modes

broadly defined, an “addressing mode” is a way of specifying an operand

RISC-V addressing modes

- **register**
e.g. all operands in “add s1, s2, s3”
- **immediate**
e.g. last operand in “addi s1, s2, 42”
- **base**
e.g. last operand in “lw s3, 496(s4)”
- **PC-relative**
e.g. last operand in “bne s0, s1, xyz”

RISC (“reduced instruction set computer”) versus CISC (“complex instruction set computer”)

RISC characterized by typical use of:

- **load-store** (register-register) style
- **fixed-length instructions**
- **limited addressing modes**
- **for basic operations, simpler instructions** that require less work to accomplish and are more easily pipelined

CISC much less well-defined than RISC, but generally:

- memory-memory or register-memory
- variable instruction length
- many instruction formats, operations, addressing modes

RISC overwhelmingly favored approach now, although Intel x86 architecture lives on ...

- fast implementations can be achieved by breaking complex “CISC” instructions into RISC-like “micro-ops”
- however, number of processors used for PCs/servers is dwarfed by the number used for other purposes (smartphones, IoT devices, ...); [here](#) it is claimed that (as of 2019) 99% of 32-bit and 64-bit processors are RISC